A CASE-STUDY BASED REPORT

ON

NETWORK INTRUSION DETECTION USING RANDOM FOREST AND GRADIENT-BOOSTED TREES

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Name Anand Akshita **roll no: 2520030043**

Name Manasvi Param **roll no: 2520030072**

Under the Esteemed guidance of

Mr. Shaik Asif

Asst.Professor, Dept. of CSE



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract **3**

Introduction **4**

Background 4
Motivation for the Project 4
Overview of the Work Carried Out 4
Scope and Responsible-Use Statement 5

Problem Statement **5**

Case-Study Scenario 5
Central Case-Study Question 6
Formal Problem Definition 6
Objectives that Guide the Study 6
Assumptions and Limitations 7

Dataset and Exploratory Data Analysis **7**

About the Dataset 7
Purpose of EDA 8
Preparation Steps Actually Performed 8
EDA Exploration - Implementation 9
Summary of EDA Findings 12

Algorithm Selection and Implementation **12**

Conclusion **16**

References **17**

Signature of the Team members

Abstract

A signature-based intrusion detection system (IDS) can only recognise attacks it has already seen. This case study follows Meera, a security analyst in a Security Operations Center, who is asked whether network attacks can be caught using data rather than a list of known signatures. We turn her question into a supervised classification problem on the NSL-KDD benchmark (148,517 connection records, split 70/15/15 into 103,801 training, 22,172 validation and 22,544 test records) and compare two tree-ensemble models: Random Forest (bagging) and Gradient-Boosted Trees (boosting).

Exploratory analysis showed a severe class imbalance (U2R is only 0.04% of the training data, a 1,206 : 1 ratio against Normal), one constant feature, several near-duplicate features ($|r| > 0.95$) and a service field with 69 categories. After dropping the constant feature, encoding the three categorical features and standardising the 37 numeric ones, both models were trained on 40 features for the Normal-versus-Attack task. Both scored above 99.8% on the validation split, but on the held-out test split, which contains 17 attack types never seen in training, accuracy fell to 77.7% (Random Forest) and 81.8% (Gradient-Boosted Trees). Gradient-Boosted Trees caught more attacks (recall 70.2% vs 63.0%) at about the same false-alarm level, while the ROC-AUC of the two models was almost identical (0.966 vs 0.965). DoS and Probe attacks were detected almost perfectly, but R2L and U2R attacks were almost entirely missed.

The main lesson is an honest one: ensemble models give a systematic, behaviour-based way to flag suspicious traffic, but they cannot be trusted on rare or novel attack types without more examples, and accuracy alone hides this weakness.

Introduction

Background

Computer networks now carry almost every service an organisation depends on, and every connection leaves a trace: the protocol used, the service requested, how long it lasted, how many bytes moved in each direction and whether it ended normally. A Network Intrusion Detection System (NIDS) watches this traffic and raises an alert when something looks harmful, such as a denial-of-service flood, a port scan or an attempt to gain unauthorised access.

Traditional NIDS tools use misuse (signature) detection: traffic is compared with a database of known attack patterns. This is fast and accurate for familiar threats, but it cannot recognise a new variant whose signature does not yet exist, and the rule set has to be updated by hand. Anomaly-based detection, in contrast, learns what normal and malicious behaviour look like from data, and so has the potential to generalise to attacks it was not explicitly told about [1].

Research on learned detectors has relied heavily on the KDD Cup 1999 data. Tavallaee et al. showed that this data set has serious statistical flaws: about 78% of its training records and 75% of its test records are duplicates, which biases learners towards frequent records and inflates results. They proposed NSL-KDD, which removes the duplicates and re-samples the records by difficulty. Abrar et al.later benchmarked eight classifiers on NSL-KDD with four different feature subsets and reported that tree-based models (Random Forest, Extra-Trees and Decision Tree) performed best, above 99% on the attack classes they tested, while U2R remained the hardest class because of its few examples. This project builds on those two papers.

Motivation for the Project

Three practical problems motivate the work:

- **Unknown attacks.** The legacy IDS in our scenario missed a new denial-of-service variant, and it took hours of manual log review before anyone noticed.
- **Too many signals for a human.** An analyst can follow instinct and known red flags (a traffic spike, an unfamiliar service, repeated failed logins), but no one can weigh dozens of such signals across thousands of connections a day.
- **Rare events are the dangerous ones.** The attack types that matter most (privilege escalation, remote break-ins) are also the rarest in the data, so it is important to find out honestly how well a model handles them, not just how well it performs on average.

Random Forest and Gradient-Boosted Trees were chosen because both are ensembles of decision trees: they combine many weak trees into one strong model, reduce overfitting compared with a single tree, work directly on mixed numeric and categorical features, and are strong, well-documented baselines for NSL-KDD.

Overview of the Work Carried Out

The work follows the standard machine-learning workflow and was carried out in Google Colab using Python, pandas, NumPy, Matplotlib/Seaborn and scikit-learn:

1. Framed the security question as a case-based scenario and a formal classification problem.

2. Loaded the three NSL-KDD splits (train 70%, validation 15%, test 15%) and checked their quality.
3. Performed exploratory data analysis (EDA): class balance, categorical features, correlations and how features behave in each attack family.
4. Prepared the data: grouped the 22 attack names into four families, dropped a constant feature, encoded categorical features and scaled numeric ones.
5. Trained a Random Forest and a Gradient-Boosted Trees classifier on the Normal-versus-Attack task.
6. Checked both models on the validation split, then evaluated them once on the test split using Accuracy, Precision, Recall, F1-score, the Confusion Matrix and ROC-AUC.
7. Looked at per-class results (Normal, DoS, Probe, R2L, U2R) and feature importance to understand where the models succeed and fail.

Scope and Responsible-Use Statement

Scope. The study covers offline, connection-level classification on the NSL-KDD data. The main task is binary (Normal vs Attack); a five-class view (Normal, DoS, Probe, R2L, U2R) is used to examine the rare classes. Two algorithms are compared with fixed, reasonable settings. Real-time deployment, packet-payload inspection, hyper-parameter search, other data sets and other algorithms are outside the scope.

Responsible use. NSL-KDD is a public benchmark derived from simulated 1998 network traffic; it contains no personal or user-content data. The results show how the methods behave on this benchmark and must not be read as a guarantee of performance on modern production networks. Any real deployment would need current data, and the model should support a human analyst, never replace one: a missed attack (false negative) and a false alarm (false positive) both have real costs, and the rare-class results reported here show why. This report describes attacks only at the level of categories and does not provide attack tools or instructions.

Problem Statement

Case-Study Scenario

It is a routine Tuesday shift for Meera, a security analyst in the Security Operations Center (SOC) of a mid-sized company. Her manager asks a simple but high-stakes question: "Can we catch an attack before it causes damage, using data rather than a list of known signatures?"

The team's legacy IDS flags traffic only when it matches a known signature. It works for familiar threats, but weeks ago it missed a new denial-of-service variant, and hours of manual log review passed before anyone noticed. Meera has a large labelled history of network connections, the NSL-KDD data set, with over 148,000 records that describe protocol type, service, duration, byte counts and error rates.

Meera treats "Is this connection normal or an attack?" as a classification task and, for attacks, also asks which of four families it belongs to: Denial of Service (DoS), Probing, Remote-to-Local (R2L) or User-to-Root (U2R). She chooses features that mirror how an analyst

reasons about a connection: the protocol and service, how long it lasted, how many bytes moved each way, how often connections to the same host or service ended in errors, and whether a user logged in. She then trains two models: a Random Forest, which combines the votes of many independent trees, and Gradient-Boosted Trees, which build trees one after another so that each corrects the mistakes of the previous ones.

When she tests both models on connections they have never seen, including attack types absent from training, the result is a humbling but honest one: both catch nearly all DoS and Probe attacks, but accuracy falls well below the near-perfect validation scores, and only a small fraction of R2L and almost no U2R attacks are caught. Gradient-Boosted Trees edge out Random Forest overall, yet neither model can be trusted on the rarest attacks without more examples. This report reproduces Meera's workflow and checks that outcome with real numbers.

Central Case-Study Question

Can a machine-learning model trained on labelled past connections separate normal traffic from attacks (and attacks into their families) well enough, including on attack types it has not seen, to complement a signature-based IDS, and does boosting (Gradient-Boosted Trees) or bagging (Random Forest) do this better?

Formal Problem Definition

Let each network connection be described by a feature vector x with 40 features (3 categorical features encoded as integers and 37 numeric features). We have a labelled training set D_train of 103,801 connections, a validation set D_val of 22,172 and a test set D_test of 22,544.

- **Task A (main task, binary):** learn a function f(x) → {Normal, Attack} from D_train. Any connection whose label is not "normal" is an Attack.
- **Task B (diagnostic, five-class):** learn g(x) → {Normal, DoS, Probe, R2L, U2R} to reveal how each attack family is handled.
- **Model selection protocol:** the validation set is used only as a sanity check; the test set is used once, at the end, for the final comparison.
- **Evaluation:** Accuracy, Precision, Recall, F1-score, Confusion Matrix and ROC-AUC. Because the data are imbalanced, per-class Precision/Recall/F1 are reported as well and accuracy is never used alone.

Objectives that Guide the Study

1. Understand the NSL-KDD data through EDA: its structure, quality, class balance and the behaviour of key features in each attack family.
2. Prepare the data correctly, with no information leaking from the validation or test sets into training.
3. Implement a Random Forest and a Gradient-Boosted Trees classifier on identical data and features.
4. Compare them fairly on the same test set using several metrics, not accuracy alone.

5. Find out how well each model detects the rare attack families (R2L and U2R) and previously unseen attack types.
6. Answer the manager's question honestly: what can and cannot be trusted, and what is needed to improve it.

Assumptions and Limitations

Assumptions

- The labels in NSL-KDD are correct and each record is an independent connection.
- The provided 70/15/15 splits are used as they are, with no re-splitting: the 125,973 training and validation records together match the official KDDTrain+ file and the 22,544 test records match KDDTest+.
- Connection-level features are enough to separate normal and attack traffic; no packet payloads are used.

Limitations

- **Old, simulated data.** Results describe NSL-KDD only and may not carry over to current networks.
- **Different test distribution.** The test split contains 3,750 records (16.6%) from 17 attack names that never appear in training, and has a much larger R2L share (9.8% vs 0.8% in training). This is realistic but makes the test deliberately harder than the validation split.
- **No tuning and a single run.** Both models use sensible fixed settings (random_state = 42), with no grid search or cross-validation, so very small differences (for example in ROC-AUC) are not conclusive.
- **Rare classes.** With only 46 U2R training records, no model can learn this class reliably.
- **Per-class view limited to one model.** The five-class report was produced with the Random Forest only, and the 3,750 novel-attack test records have no family label in our mapping, so they are left out of the five-class report (they are included in the binary evaluation as attacks).
- **Label encoding** gives categorical values an arbitrary order. Tree models cope well with this, but distance-based models would need a different encoding.

Dataset and Exploratory Data Analysis

About the Dataset

NSL-KDD [1] is a refined version of the KDD Cup 1999 intrusion data, built from DARPA 1998 tcpdump traffic. Each record describes one network connection with **41 features and a label** (42 columns in the CSV files). The features fall into four groups:

- **Basic features:** protocol_type (tcp/udp/icmp), service (69 values in our training split), flag (11 connection-status values), duration, src_bytes, dst_bytes, land, wrong_fragment, urgent.

- **Content features:** hot, num_failed_logins, logged_in, root_shell, num_root, num_compromised and others that capture what happens inside a connection (important for R2L and U2R attacks).
- **Time-window traffic features (last 2 seconds):** count, srv_count, serror_rate, rerror_rate, same_srv_rate, diff_srv_rate and related rates.
- **Host-based traffic features:** dst_host_count, dst_host_srv_count, dst_host_same_srv_rate and related rates.

The label holds the attack name: 23 distinct values in the training split (normal and 22 attacks), which we group into the standard four families: **DoS** (e.g. neptune, smurf, back), **Probe** (e.g. ipsweep, nmap, portsweep, satan), **R2L** (e.g. guess_passwd, warezclient, ftp_write) and **U2R** (e.g. buffer_overflow, rootkit, perl).

Table 1. The three data splits and their class make-up

Split	Rows	Cols	Normal	DoS	Probe	R2L	U2R	Attack names not in training
Train (70%)	103,801	42	55,490 (53.46%)	37,890 (36.50%)	9,574 (9.22%)	801 (0.77%)	46 (0.04%)	-
Validation (15%)	22,172	42	11,853 (53.46%)	8,037 (36.25%)	2,082 (9.39%)	194 (0.87%)	6 (0.03%)	-
Test (15%)	22,544	42	9,711 (43.07%)	5,741 (25.47%)	1,106 (4.91%)	2,199 (9.75%)	37 (0.16%)	3,750 records (16.63%), 17 names

Two things stand out in Table 1. First, the training and validation splits have almost identical class proportions (they are drawn from the same source), which explains why validation scores are so high. Second, the test split looks different: it has fewer Normal and DoS records, a far larger share of R2L, and 3,750 records whose attack names (for example mscan, apache2 and processtable) never appear in training. The test split therefore plays the role of "connections the model has never seen" in Meera's story.

Purpose of EDA

EDA was performed before any model was trained, to answer the practical questions that decide how the models must be built and judged:

- Is the data clean (missing values, duplicates, constant columns)?
- How balanced are the classes, and which metrics will be honest under imbalance?
- What do the categorical features look like, and how should they be encoded?
- Are some numeric features redundant (highly correlated), and which are skewed?
- Do the attack families behave differently enough in the features for a model to separate them?

Preparation Steps Actually Performed

Table 2 lists exactly what was done in the notebooks, in order, and what was deliberately not done.

Table 2. Data preparation pipeline

	Step	What was done	Result
1	Load data	Read KDD_train_70.csv, KDD_val_15.csv and KDD_test_15.csv with pandas.	103,801 / 22,172 / 22,544 rows; 42 columns each
2	Quality checks	Counted missing values and duplicate rows; inspected data types.	0 missing values and 0 duplicate rows; 23 integer, 15 float and 4 text columns (protocol_type, service, flag, label)
3	Group labels	Mapped the 22 attack names to DoS, Probe, R2L or U2R (column attack_category).	No unmapped labels in train/validation; 3,750 test records with unseen attack names stay unmapped
4	Drop constant feature	Found features with a single value.	num_outbound_cmds removed; 40 features remain
5	Binary target	Label "normal" = Normal, anything else = Attack.	Target for the main task
6	Encode categoricals	LabelEncoder on protocol_type, service and flag, fitted on the training split only. Values not seen in training are mapped to an "unseen" bucket.	Pipeline does not crash on new categories
7	Scale numeric features	StandardScaler fitted on the training split only, then applied to validation and test.	37 numeric features with mean 0, std 1; no test information leaks into training
8	Keep given splits	No re-splitting was done.	Validation = sanity check; test = final evaluation

Not done: no resampling (such as SMOTE), no feature selection and no hyper-parameter search were applied. Class imbalance was handled only through class_weight="balanced" in the Random Forest. These are listed as future work.

EDA Exploration - Implementation

1. Class distribution

```
attack_map = {'normal': 'Normal', 'neptune': 'DoS', 'smurf': 'DoS', ... 'rootkit': 'U2R'}
for df in (train, val, test):
    df['attack_category'] = df['label'].map(attack_map)
cat_counts = train['attack_category'].value_counts()
```

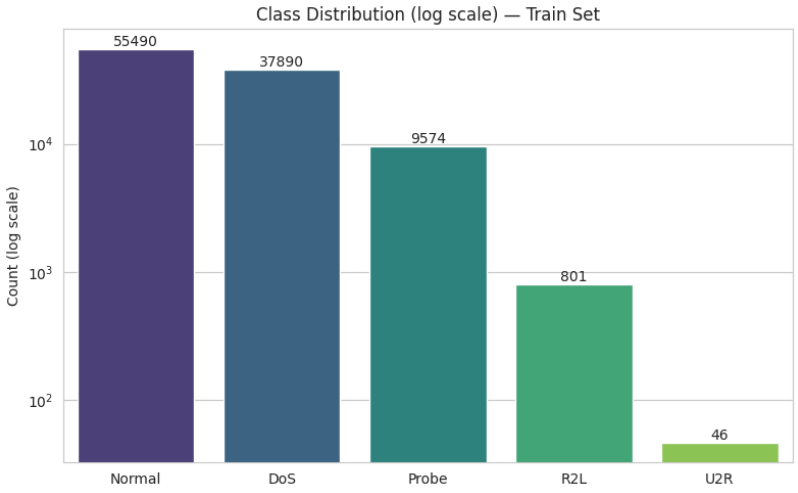


Figure 1. Class distribution of the training split (log scale)

Normal (53.46%) and DoS (36.50%) dominate. R2L has 801 records (0.77%) and U2R only 46 (0.04%); the Normal : U2R ratio is about 1,206 : 1. A model that labelled every U2R connection as Normal would still look almost perfect on overall accuracy, so recall and F1 per class must be examined.

2. Categorical features

```
for col in ['protocol_type', 'flag']:
    print(train[col].value_counts(normalize=True) * 100)
print(train['service'].nunique())      # 69 categories
```

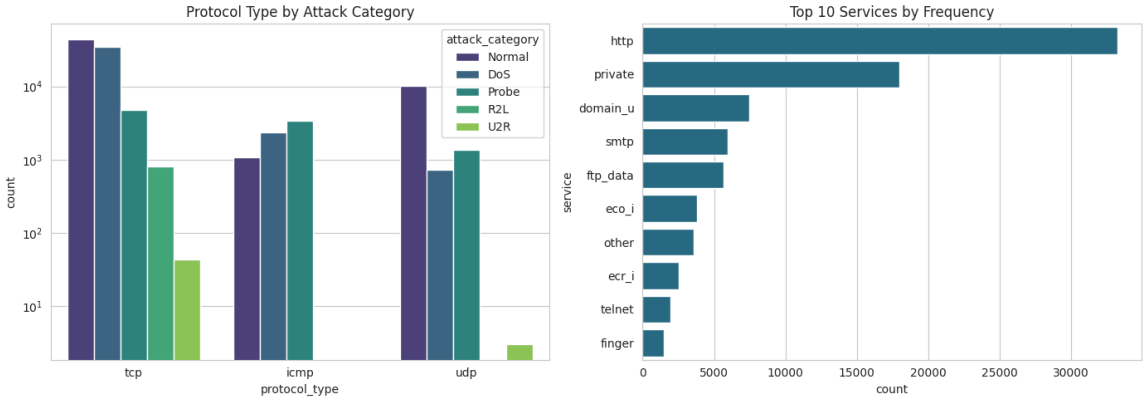


Figure 2. Protocol type by attack category (left) and the ten most frequent services (right)

TCP carries 81.5% of connections, UDP 11.9% and ICMP 6.6%. The most frequent flag is SF (normal completion, 59.4%), followed by S0 (connection attempt with no reply, 27.7%) and REJ (rejected, 8.9%). The service field has 69 categories, led by http (33,263 records), private, domain_u, smtp and ftp_data. Protocol already separates families: 91.9% of DoS records are TCP, while 35.6% of Probe records are ICMP compared with 2.0% of Normal records.

3. Correlated numeric features

```
corr = train[numeric_cols].corr()
pairs = [(a, b, corr.loc[a, b]) for a, b in combinations(numeric_cols, 2)
         if abs(corr.loc[a, b]) > 0.95]
```

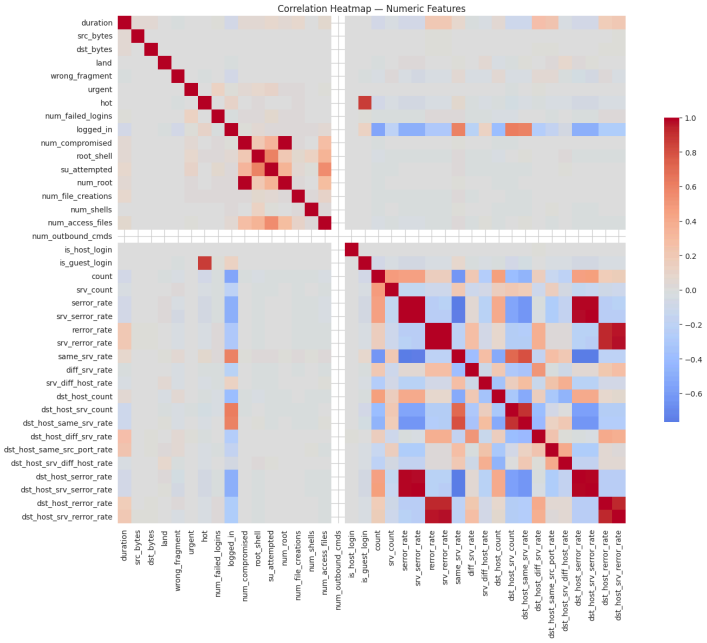


Figure 3. Correlation heatmap of the numeric features

Table 3. The ten feature pairs with |correlation| > 0.95

Feature 1	Feature 2	Correlation
num_compromised	num_root	0.999
serror_rate	srv_serror_rate	0.993
rerror_rate	srv_rerror_rate	0.990
srv_serror_rate	dst_host_srv_serror_rate	0.986
dst_host_serror_rate	dst_host_srv_serror_rate	0.985
serror_rate	dst_host_srv_serror_rate	0.981
serror_rate	dst_host_serror_rate	0.979
srv_serror_rate	dst_host_serror_rate	0.978
srv_rerror_rate	dst_host_srv_rerror_rate	0.971
rerror_rate	dst_host_srv_rerror_rate	0.965

Ten pairs are almost duplicates. The error-rate features (serror/rerror at connection, service and host level) form clusters that carry nearly the same information. Tree ensembles are not harmed by this, but distance-based or linear models would benefit from removing redundant columns.

4. Feature behaviour by attack family

```
sns.boxplot(data=train, x='attack_category', y=np.log1p(train['src_bytes']), order=order)
sns.boxplot(data=train, x='attack_category', y='serror_rate', order=order)
```

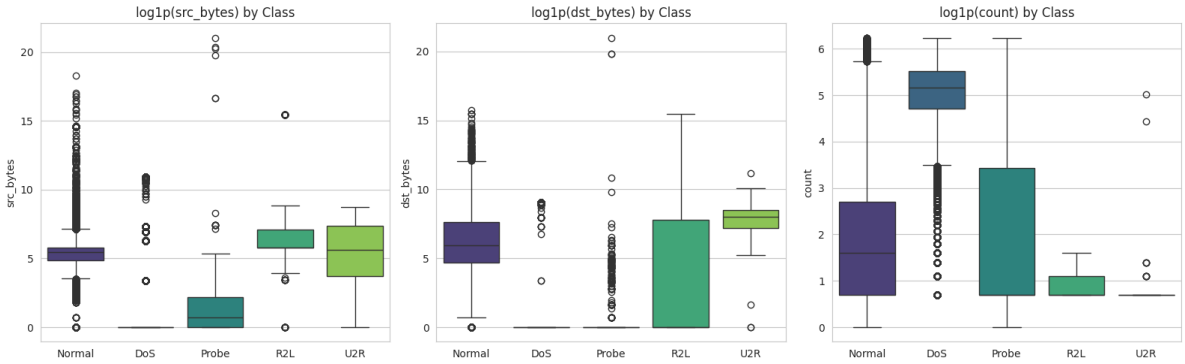


Figure 4. log(1 + value) of src_bytes, dst_bytes and count for each attack category

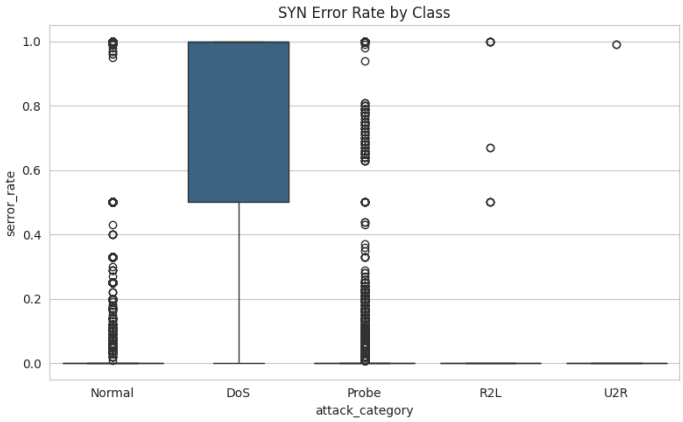


Figure 5. SYN error rate (serror_rate) by attack category

DoS connections look very different from Normal ones: the average serror_rate is 0.75 for DoS against 0.01 for Normal, the median connection count in the 2-second window is 173 against 4, and the median src_bytes and dst_bytes are 0 (floods carry no payload) against 233 and 378 for Normal. Probe connections also have almost no payload and a median count of 1. R2L and U2R, however, are logged-in sessions (logged_in is 0.92 for R2L and 0.87 for U2R, against 0.71 for Normal) with normal-looking error rates; they hide inside ordinary sessions, which foreshadows how hard they are to detect.

Summary of EDA Findings

Table 4. EDA findings and what they mean for the models

Finding	Implication for modelling
Data are clean: 0 missing values and 0 duplicate rows	No imputation or de-duplication needed
Severe imbalance: U2R 46 records (0.04%), R2L 801 (0.77%)	Report per-class Precision/Recall/F1, never accuracy alone; consider class weights or resampling
Test split contains 17 attack names absent from training and a very different R2L share	Expect a drop from validation to test; validation score alone is over-optimistic
num_outbound_cmds is constant	Dropped (no information)
service has 69 categories	Label-encode for trees; one-hot would add 69 columns
Ten feature pairs with |r| > 0.95	Tolerated by tree ensembles; remove or combine for other model types
Skewed byte and count features (very wide range)	Standardise; tree models are insensitive to scale

Finding	Implication for modelling
DoS and Probe differ strongly from Normal (error rates, counts, bytes, protocol)	These families should be easy to detect
R2L and U2R look like ordinary logged-in sessions	These families are likely to be hard to detect

Algorithm Selection and Implementation

Why tree ensembles, and why these two

A single decision tree is easy to understand but tends to overfit. An ensemble combines many trees so that individual mistakes cancel out. The two main ways to build such an ensemble are bagging and boosting, and the case study asks which of them suits intrusion detection better.

Table 5. Why the two algorithms were selected

	Random Forest (bagging)	Gradient-Boosted Trees (boosting) [5]
Idea	Many deep trees trained independently on random samples of rows and features; the majority vote is the answer [4]	Many shallow trees trained one after another; each new tree is fitted to the errors left by the previous ones
Main strength	Stable, hard to overfit, little tuning, trains in parallel	Can capture subtle, non-obvious feature combinations; often more accurate
Main risk	May miss fine decision boundaries	More sensitive to settings and noise; trains sequentially (slower)
Imbalance handling here	class_weight = "balanced"	No class weighting in this run

Both operate directly on the encoded mixed features, need no feature scaling to work (scaling was kept so the same pipeline can serve other models later), and are the strongest families in the NSL-KDD benchmark study [2]. Note that the two models were not configured identically on imbalance: only the Random Forest used class weights. This is a limitation of the comparison and is discussed below.

Implementation

Table 6. Model settings

Setting	Random Forest	Gradient-Boosted Trees
scikit-learn class	RandomForestClassifier	GradientBoostingClassifier
Number of trees	200	200
Tree depth	unlimited (max_depth = None)	3
Learning rate	not applicable	0.1
Class weighting	balanced	none
Random seed	42	42
Training time (Colab)	26.3 s	51.3 s

```
rf_model = RandomForestClassifier(n_estimators=200, max_depth=None, n_jobs=-1,
                                  random_state=42, class_weight='balanced')
rf_model.fit(X_train, y_train_bin)

gbt_model = GradientBoostingClassifier(n_estimators=200, learning_rate=0.1,
                                       max_depth=3, random_state=42)
gbt_model.fit(X_train, y_train_bin)
```

The inputs are the 40 prepared features (X_train, 103,801 rows) and the binary target (1 = Attack). After training, both models were first checked on the validation split. They scored almost perfectly: Random Forest accuracy 0.9989 and F1 0.9988; Gradient-Boosted Trees accuracy 0.9982 and F1 0.9980. These figures describe data that resembles the training data, so they are only a sanity check. The test split was then used once for the final evaluation.

Evaluation on the held-out test split

The test split has 22,544 connections, of which 12,833 (56.9%) are attacks and 9,711 are normal.

Table 7. Test-set results, Normal vs Attack

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Random Forest	0.7770	0.9671	0.6297	0.7628	0.9662
Gradient-Boosted Trees	0.8176	0.9692	0.7019	0.8142	0.9653

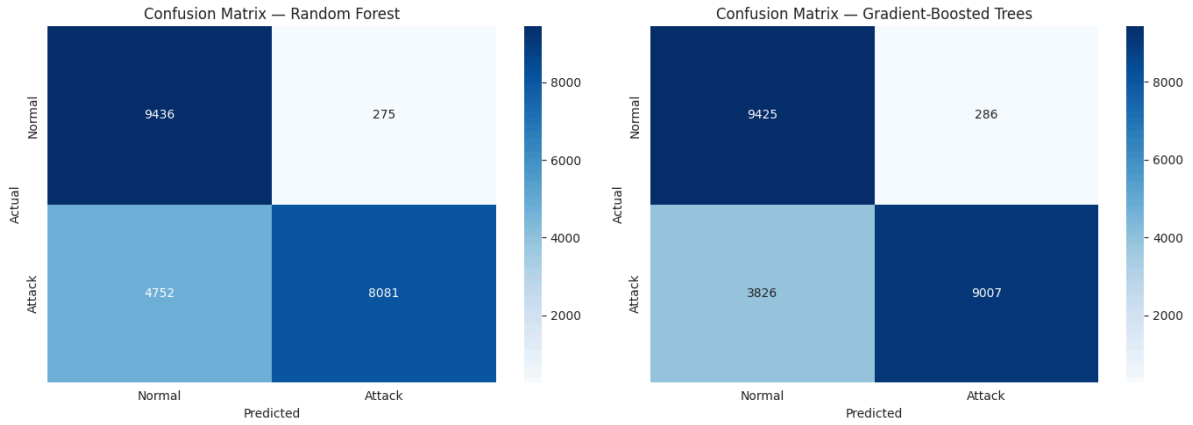


Figure 6. Confusion matrices on the test split (rows = actual, columns = predicted)

Table 8. Confusion-matrix counts from Figure 6

	Normal correctly passed (TN)	False alarms (FP)	Attacks missed (FN)	Attacks caught (TP)
Random Forest	9,436	275	4,752	8,081
Gradient-Boosted Trees	9,425	286	3,826	9,007

- **Precision is high for both (about 0.97).** When either model raises an alert, it is almost always a real attack. False alarms are low: 275 (2.8% of Normal) for Random Forest and 286 (2.9%) for Gradient-Boosted Trees.
- **Recall is where they differ.** Gradient-Boosted Trees caught 9,007 of 12,833 attacks (70.2%) and Random Forest 8,081 (63.0%), so boosting found 926 more attacks for 11 more false alarms. Its F1-score is higher (0.8142 vs 0.7628) and so is its accuracy (81.8% vs 77.7%).
- **Ranking quality is almost the same.** ROC-AUC is 0.9662 for Random Forest and 0.9653 for Gradient-Boosted Trees (Figure 7). Both models can rank attacks above normal traffic very well; the gap in recall comes from where the default decision threshold falls, not from a big difference in ability. With a single run and no cross-validation, this tiny AUC difference should not be treated as meaningful.

- **Even the better model misses nearly 30% of attacks**, so neither can replace the signature-based IDS.

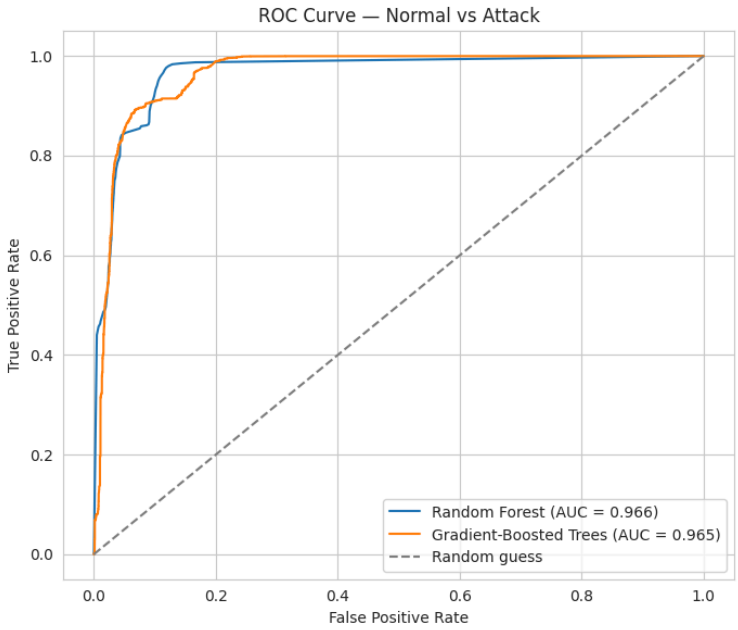


Figure 7. ROC curves of both models for Normal vs Attack

Per-class view

The binary result hides which attacks are missed. Table 9 shows a five-class Random Forest, trained on the same features, evaluated on the 18,794 test records whose attack names belong to the four families.

Table 9. Five-class Random Forest, per-class test results

Class	Precision	Recall	F1-score	Support
DoS	0.988	0.991	0.989	5,741
Normal	0.808	0.972	0.883	9,711
Probe	0.823	1.000	0.903	1,106
R2L	0.875	0.003	0.006	2,199
U2R	0.000	0.000	0.000	37
Overall accuracy			0.864	18,794
Macro average	0.699	0.593	0.556	18,794
Weighted average	0.870	0.864	0.812	18,794

DoS and Probe, the two well-represented attack families, are caught almost perfectly (recall 0.991 and 1.000). R2L has a recall of only 0.003 and U2R of zero. The overall accuracy of 0.864 here looks respectable only because DoS and Normal make up most of the records; the macro-average F1 of 0.556 shows the real picture. This matches the case scenario: the models are strong where there are many examples and weak where there are few.

Which features drive the decision?

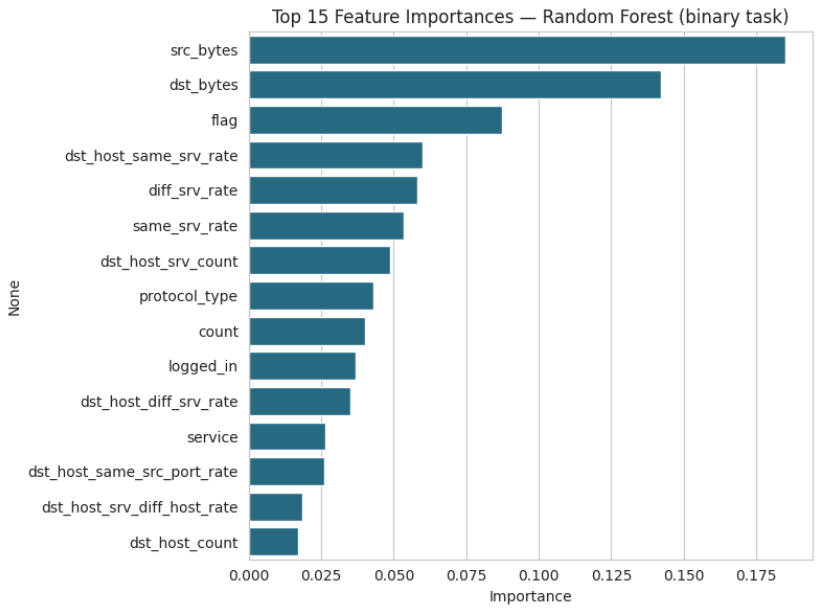


Figure 8. Top 15 feature importances, Random Forest (binary task)

The most important features are src_bytes and dst_bytes, followed by flag, dst_host_same_srv_rate, diff_srv_rate, same_srv_rate, dst_host_srv_count, protocol_type and count. This agrees with the EDA and with analyst intuition: how much data moved, how the connection ended, and how concentrated or scattered the connections are on a service. Flood attacks and scans leave exactly these traces.

Why did accuracy fall from 99.9% to 78-82%?

The gap between validation and test is the main finding of the study. Three facts from the data explain it:

1. **Novel attacks.** 3,750 test records (16.6% of the test split, 29% of all test attacks) belong to 17 attack names that were never in the training data. A model cannot learn a pattern it has never been shown.
2. **A different mix.** R2L is 0.8% of the training data but 9.8% of the test data, so the weakest class is weighted far more heavily in the test.
3. **Attacks that look normal.** R2L and U2R connections are logged-in sessions with normal-looking error rates (see the EDA findings), so even the known ones are hard to separate from normal traffic. This is the most likely explanation for the near-zero R2L/U2R recall, consistent with the EDA, although we did not run a separate experiment to prove it.

The original project plan (presentation) expected both models to exceed 99% accuracy. That holds on the validation split but not on the test split, and reporting the lower figure is the honest answer to the case-study question.

Answer to the case-study question

- **Does ML help?** Yes, for the common attack families. Both models detect DoS and Probe attacks almost perfectly from behaviour alone, with no signature, and raise very few false alarms (precision about 0.97).
- **Which model is better?** On this test split Gradient-Boosted Trees is better overall (higher accuracy, recall and F1, fewer missed attacks) while Random Forest trains about twice as fast and has an equal ROC-AUC. Because the models were not configured identically (class weights on Random Forest only), a fully fair comparison would repeat the experiment with matched settings and cross-validation.
- **Can it be trusted on rare and novel attacks?** No. R2L, U2R and unseen attack types need more examples, resampling or additional detection methods before this model could be relied on.

Conclusion

This case study turned a security manager's question into a complete machine-learning workflow on the NSL-KDD data: framing the problem, exploring the data, preparing it without leakage, training two tree-ensemble classifiers and evaluating them honestly.

What we found. The data are clean but strongly imbalanced. Both Random Forest and Gradient-Boosted Trees reach more than 99.8% on validation data that resembles the training data, but on the test split, which contains attack types unseen in training, accuracy falls to 77.7% and 81.8% respectively. Gradient-Boosted Trees is the better overall model (recall 0.7019 vs 0.6297, F1 0.8142 vs 0.7628) with an almost identical false-alarm rate and ROC-AUC (0.9653 vs 0.9662). Both models detect DoS and Probe attacks almost perfectly, while R2L and U2R attacks are almost completely missed.

What this means for Meera's manager. A data-driven detector is a useful complement to a signature-based IDS: it finds the high-volume, behaviour-heavy attacks without needing their signatures, and when it raises an alarm it is usually right. It is not yet a replacement: roughly three in ten attacks in the test set slipped past even the better model, and the subtle attacks that cause the most harm are exactly those it misses. Accuracy alone would have hidden this, which is why per-class recall, F1 and the confusion matrix were used.

Limitations. The data are old and simulated, the two models were not configured identically on class imbalance, no tuning or cross-validation was done, and the per-class analysis covers one model.

Future work.

- Handle imbalance for R2L and U2R: class weights for both models, SMOTE or other resampling, and more examples.
- Tune hyper-parameters (grid or random search with cross-validation) and compare against XGBoost or LightGBM.

- Lower the decision threshold, or add a separate anomaly detector, to trade a few more false alarms for higher recall.
- Remove redundant features and test feature selection to cut computation time.
- Test on newer data sets and extend to real-time and wireless/mobile networks.

References

[1] M. Tavallaee, E. Bagheri, W. Lu and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," in Proc. IEEE Symposium on Computational Intelligence in Security and Defense Applications (CISDA), 2009.

[2] I. Abrar, Z. Ayub, F. Masoodi and A. M. Bamhdi, "A Machine Learning Approach for Intrusion Detection System on NSL-KDD Dataset," in Proc. International Conference on Smart Electronics and Communication (ICOSEC), 2020.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[4] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[5] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.